

BOOT4DIM

부트사차원

C언어 중급 교육

PRESENTER : 최민혁

1회차

2026. 3.17 (화)



CONTENTS

포인터와 메모리 주소의 이해

01

포인터의 개념



04

포인터와 배열의 관계



02

포인터의 선언과 참조



05

총정리



03

Call By Value, Call By Reference



06





포인터 변수: 어떤 변수의 **포인터 값**을 저장하는 변수

포인터 값: 어떤 변수의 **물리적 주소**, **변수형**을 포인터 값이라고 한다.



```
int x = 10;
```

```
int* ptr = &x;
```

*(애스터리스크) 이용하여 포인터 변수를 선언한다.

포인터 변수는 Integer 타입의 변수x의 포인터 값을 저장한다.



포인터 변수: 어떤 변수의 **포인터 값**을 저장하는 변수

포인터 값: 어떤 변수의 **물리적 주소**, **변수형**을 포인터 값이라고 한다.



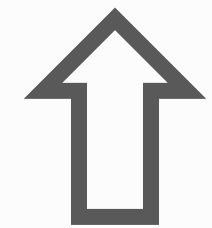
```
int x = 10;  
int* ptr = &x;  
printf("%d", *ptr);
```

* 연산자를 이용하여 포인터 변수에 저장된 값을 참조한다.



포인터 변수: 어떤 변수의 **포인터 값**을 저장하는 변수

포인터 값: 어떤 변수의 **물리적 주소**, **변수형**을 포인터 값이라고 한다.



왜 저리 복잡하게 정의해 놓았을까?

주소를 통한 실제 공간 접근 + 변수형을 통해 변수의 길이, 해석법 결정



```
int* ptr1; //0x100부터 4바이트 읽어서 int로 해석  
char* ptr2; //0x200부터 1바이트 읽어서 char로 해석
```

ptr1 FF

ptr2 F2

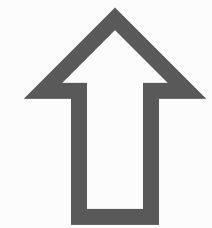
FF	0	0	0	10
----	---	---	---	----

F2	0	0	0	41
----	---	---	---	----



포인터 변수: 어떤 변수의 **포인터 값**을 저장하는 변수

포인터 값: 어떤 변수의 **물리적 주소**, **변수형**을 포인터 값이라고 한다.



왜 저리 복잡하게 정의해 놓았을까?

만약 ptr1에 F2의 값을 읽게 한다면?



```
int* ptr1; //0x100부터 4바이트 읽어서 int로 해석  
char* ptr2; //0x200부터 1바이트 읽어서 char로 해석
```

ptr1 FF

FF	0	0	0	10
----	---	---	---	----

ptr2 F2

F2	0	0	0	41
----	---	---	---	----



포인터 변수: 어떤 변수의 **포인터 값**을 저장하는 변수

포인터 값: 어떤 변수의 **물리적 주소**, **변수형**을 포인터 값이라고 한다.

1_locating_ptr.c

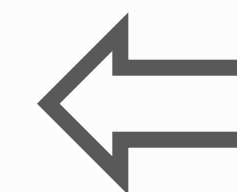
```
#include <stdio.h>

int main() {
    int x = 10;
    int* ptr = &x;

    printf("%d\n", x);
    printf("%p\n", &x);

    printf("%p\n", &ptr);
    printf("%p\n", ptr);
    printf("%d\n", *ptr);
}
```

X
ADDR: 0x100
VAL: 10



PTR
ADDR: 0x300
VAL: ptr val of X

```
10
000000000005ffe7c
000000000005ffe70
000000000005ffe7c
10
```



Call By Value

값에 의한 호출

호출자의 인자, 수신자의 매개변수는 다른 저장공간에 할당

함수 내에서의 매개변수의 변화는 원본 변수에 영향을 끼치지 않음

```
void swap(int x, int y) {  
    int temp = x;  
    x = y;  
    y = temp;  
}
```

포인터 변수: 어떤 변수의 **포인터 값**을 저장하는 변수
포인터 값: 어떤 변수의 **물리적 주소**, **변수형**을 포인터 값이라고 한다.

Call By Reference

참조에 의한 호출

수신자의 매개변수는 원본 변수의 메모리 주소를 가리킴

함수 내부에서의 매개변수 변화는 원본 변수까지 영향을 끼침

```
void swap(int* x, int* y)  
{  
    int temp = *x;  
    *x = *y;  
    *y = temp;  
}
```




Call By Value

값에 의한 호출



```
void swap(int x, int y) {  
    int temp = x;  
    x = y;  
    y = temp;  
}  
  
int main(){  
    int a = 1;  
    int b = 2;  
    swap(a, b);  
  
    printf("%d %d\n", a, b);  
}
```

포인터 변수: 어떤 변수의 **포인터 값**을 저장하는 변수

포인터 값: 어떤 변수의 **물리적 주소**, **변수형**을 포인터 값이라고 한다.





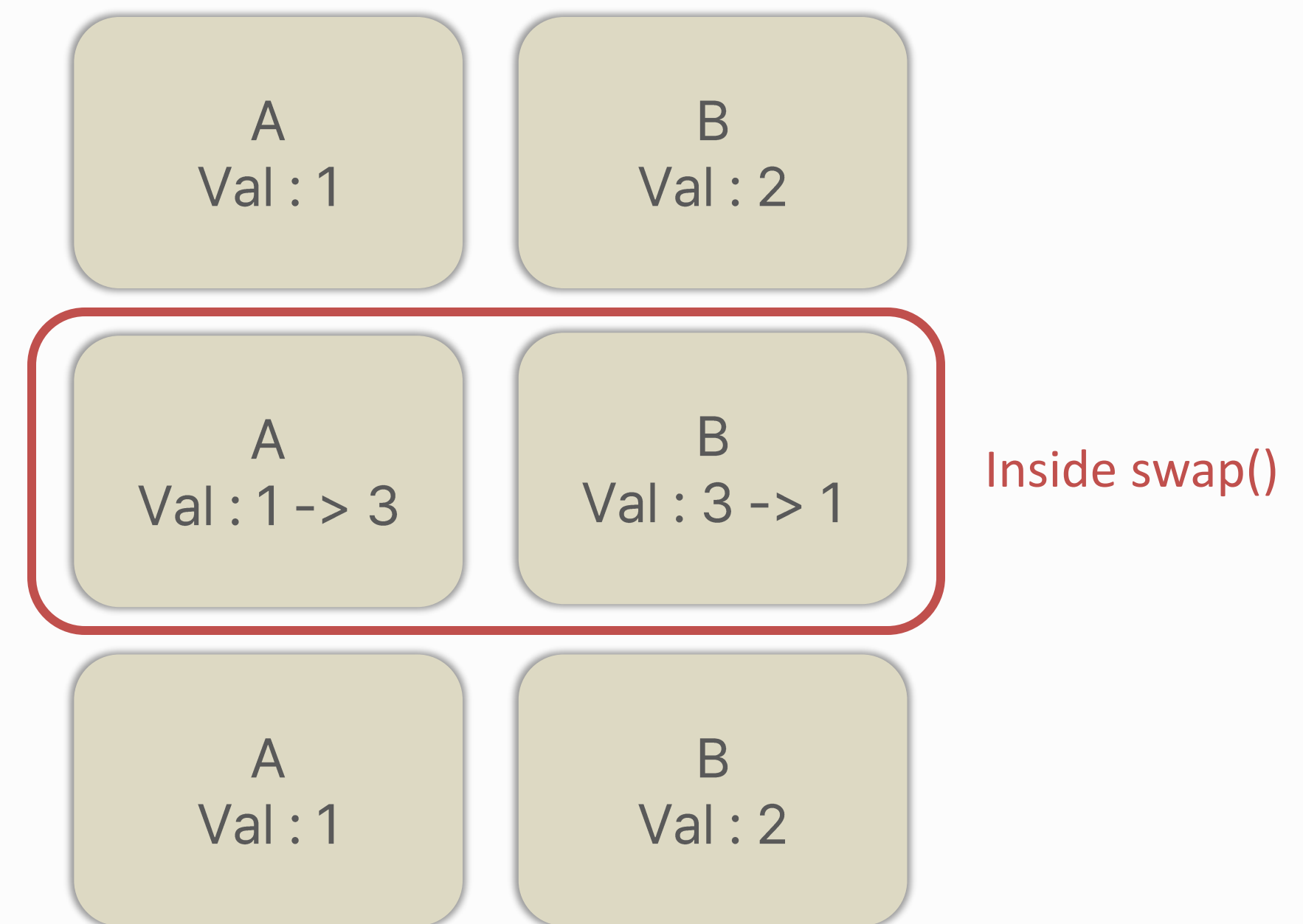
Call By Reference

참조에 의한 호출

2_swap.c

```
void swap(int* x, int* y){  
    int temp = *x;  
    *x = *y;  
    *y = temp;  
}  
  
int main(){  
    int a = 1;  
    int b = 2;  
    swap(&a, &b);  
  
    printf("%d %d\n", a, b);  
}
```

포인터 변수: 어떤 변수의 **포인터 값**을 저장하는 변수
포인터 값: 어떤 변수의 **물리적 주소**, **변수형**을 포인터 값이라고 한다.





배열의 식별자는, 해당 배열의 **첫 번째 원소의 포인터값**이다.

Call By Reference

참조에 의한 호출

```
int arr[5] = {1,2,3,4,5};
```

```
printf("%p\n", &arr);
```

```
printf("%p\n", arr);
```

```
printf("%d\n", *arr);
```

1. 배열의 식별자가 & 또는 sizeof와 사용된다면, 배열 전체의 포인터값으로 사용된다.
2. 배열의 식별자가 *와 사용된다면, 배열의 참조로 사용된다.
3. 1과 2의 경우가 아니라면, 배열의 첫 번째 원소의 포인터값으로 사용된다.



1. 배열의 식별자가 **&** 또는 **sizeof**와 사용된다면, **배열 전체의 포인터 값**으로 사용된다.

Call By Reference

참조에 의한 호출

3_sizeof.c

```
#include <stdio.h>

int main(){
    int arr[5] = {1,2,3,4,5};

    printf("%p\n", arr);
    printf("%p\n", &arr);

    printf("%d\n", sizeof(arr));
    printf("%d\n", sizeof(arr + 0));

    return 0;
}
```

1. 식별자가 단독으로 사용되었으므로 배열의 시작 주소 출력
2. 식별자가 &와 사용되었으므로 배열 전체의 포인터로 사용
3. 식별자가 sizeof 연산자와 사용되었으므로 배열 전체의 크기 $5 * 4 = 20$ 출력
4. 식별자가 +라는 연산자와 함께 사용되었으므로 첫 번째 요소의 크기 4 출력



2. 배열의 식별자가 *와 사용된다면, 배열의 참조로 사용된다.

Call By Reference

4_asterisk.c

```
#include <stdio.h>

int main(){
    int arr[5] = {1,2,3,4,5};

    printf("%d\n", arr[0]);
    printf("%d\n", *(arr + 0));
    printf("%d\n", *(arr + 1));
    printf("%d\n", *(arr + 2));
    printf("%d\n", *(arr + 3));
    printf("%d\n", *(arr + 4));

    printf("%d\n", 3[arr]);

    return 0;
}
```

$arr[n] == *(arr + n)$

arr[0]은 *(arr + 0)과 완벽히 동치이므로, 동일한 값이 나온다.

이론적으로 arr[3]을 3[arr]으로 적용할 수는 있다.



3. 1과 2의 경우가 아니라면, 배열의 첫 번째 원소의 포인터값으로 사용된다.

Call By Reference

참조에 의한 호출

5_else.c

```
#include <stdio.h>

int main(){
    int arr[5] = {1,2,3,4,5};

    printf("%p\n", arr);
    printf("%p\n", arr+1);
    printf("%p\n", arr+2);
    printf("%p\n", arr+3);
}
```

1. 배열의 식별자가 단독으로 사용되었으므로 arr의 첫 번째 주소 출력
2. 배열의 식별자가 단독으로 사용되었으므로 arr의 첫 번째 요소인 1이 출력
이후 덧셈이 사용되어도 arr은 첫 번째 원소를 가리키므로, 순차적인 주소값의 결과를 갖는다.



```
// === 미션 2: 함수 작성 ===
void upgrade_score(int *p_score) {
    //Your Code Here
}

int main() {
    int scores[5] = {80, 62, 90, 92, 74};
    int *ptr = scores;

    //=== 미션 1: 포인터 참조 통한 배열값 확인 ===
    //Your Code Here

    upgrade_score(ptr);

    //=== 미션 3: 배열 수정 이후 배열값 확인 ===
    //Your Code Here

    return 0;
}
```

[실습 문제] 포인터로 조종하는 미니 성적 관리 프로그램

1. Main 함수 내에 학생 5명의 점수를 저장하는 배열 선언 {80, 62, 90, 92, 74}
2. 함수 내에 배열을 가리키는 포인터 정의
3. Upgrade_score 함수 정의, 배열의 각 요소를 +5 하는 함수

미션1 : main함수에서 포인터를 이용하여 배열의 모든 요소 print

미션2 : Upgrade_score을 통해 모든 요소 값을 바꾸기

미션3: main함수에서 포인터를 이용하여 배열의 모든 요소 print



Q & A